

90-DAY DSA ROADMAP — Comprehensive Study Guide

From Beginner to Advanced: day-by-day breakdown, concepts, practice problems, tips, and resources

Overview & How to Use this Roadmap

- This 90-day roadmap splits topics into focused phases. Each phase lists objectives, day-by-day focus, concept summaries, example problems, and practice targets.
- Follow daily goals, solve recommended problems, review mistakes, and timebox practice sessions. Aim for consistency over intensity.
- Use at least one coding practice platform (LeetCode/Codeforces/InterviewBit) and one theory/reference (GeeksforGeeks/CP-Algorithms/CLRS).

Study Method & Schedule Template

- Daily structure (suggested): 30–45 min concept review, 60–90 min problem solving, 15–30 min review & notes.
- Use Active Recall: after reading, close notes and write the core idea + pseudocode from memory.
- Use Spaced Repetition for key patterns and algorithms.
- Maintain a 'mistakes log'—store failed problems and re-solve weekly.

Phase 1: Basics & Foundations (Day 0–20)

Goal:

Pick primary language (C++/Java/Python). Master syntax, I/O, conditionals, loops, functions, basic data types, and recursion basics.

Daily plan (example):

Days 0–3: language setup and syntax + simple exercises. Days 4–8: functions, arrays basics, loops practice. Days 9–12: strings and basic manipulations. Days 13–16: simple recursion & divide-and-conquer thinking. Days 17–20: solve 20–30 easy problems spread across topics.

Core concepts:

Variables, control flow, functions, arrays, strings, input/output, debugging, recursion base case & recursion tree.

Practice targets:

Complete 40–60 easy problems on arrays/strings/loops. Focus on clean submission and edge cases.

Sample problems:

Two Sum (easy variant), Reverse String, Remove Duplicates from Array, Palindrome check, Factorial (recursion).

Pitfalls:

Skipping rigorous debugging, not handling edge cases, not writing simple assertions/tests.

Phase 2: Time & Space Complexity (Day 1–2)

Goal:

Understand Big O / Big Theta / Big Omega notation and space analysis.

Daily plan:

Day 1: Read definitions and examples. Day 2: Analyze 10 sample algorithms (loops, nested loops, recursion).

Core concepts:

Worst, average, best-case; time vs space trade-offs; common complexities ($O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$).

Practice targets:

Analyze runtime of code snippets. Practice deriving recurrence relations for recursive solutions.

Sample problems:

Analyze iterative binary search ($O(\log n)$), merge sort recurrence $T(n)=2T(n/2)+O(n)$.

Phase 3: Arrays & Strings (Day 3–17)

Goal:

Master array manipulations and string techniques (sliding window, two pointers).

Daily plan:

Days 3–6: 1D arrays basics and two pointers. Days 7–10: 2D arrays and matrix problems. Days 11–14: string patterns, substrings, palindrome, sliding window. Days 15–17: mixed easy–medium problems.

Core concepts:

Indexing, in-place operations, two-pointer patterns, sliding window, prefix sums, frequency counts.

Practice targets:

Solve 50 problems spanning easy → medium covering two-pointer and sliding-window patterns.

Sample problems:

Two Sum, 3Sum (medium), Longest Substring Without Repeating Characters, Container With Most Water, Rotate Array.

Pitfalls:

Overlooking off-by-one errors, not practicing in-place transforms, ignoring space constraints.

Phase 4: Sorting Basics (Day 18–22)

Goal:

Understand elementary sorting algorithms and their tradeoffs.

Daily plan:

Day 18: Bubble & Insertion Sort understanding and implementation. Day 19: Selection Sort and complexity comparisons. Day 20–22: practice problems where custom sorting logic is required.

Core concepts:

Stability, in-place vs out-of-place, average/worst-case complexities, and when to use simple sorts vs advanced sorts.

Practice targets:

Implement 3 sorts from scratch and analyze cases.

Sample problems:

Sort colors (Dutch National Flag), custom comparator sorting tasks.

Pitfalls:

Assuming simple sorts are efficient on large n ; ignoring stability requirements when needed.

Phase 5: Binary Search (Day 23–29)

Goal:

Master binary search patterns and applications on monotonic functions and answer-space binary search.

Daily plan:

Day 23–24: classic binary search template. Day 25–26: binary search on rotated arrays, finding first/last occurrence. Day 27–29: binary search on answer (Aggressive Cows, Painter Partition).

Core concepts:

Invariant maintenance, mid computation $(\text{left} + \text{right})/2$ vs $\text{left} + (\text{right} - \text{left})//2$, lower_bound/upper_bound patterns.

Practice targets:

Solve 20 problems using binary search variations.

Sample problems:

Search in Rotated Sorted Array, Find First and Last Position of Element, Median of Two Sorted Arrays (intro).

Pitfalls:

Wrong mid formula causing overflow in some languages, incorrect invariants leading to infinite loops.

Phase 6: Recursion (Day 30–36)

Goal:

Build recursive reasoning and simple divide-and-conquer solutions.

Daily plan:

Days 30–31: recursion basics & tracing. Days 32–34: backtracking introduction with small recursion problems. Days 35–36: optimize recursion with memoization.

Core concepts:

Call stack, base case, recursion depth, memoization vs tabulation.

Practice targets:

Solve classic recursion problems to build intuition.

Sample problems:

Factorial, Fibonacci (memoized), Generate subsets, Permutations.

Pitfalls:

Not identifying base cases, excessive recomputation without memoization.

Phase 7: Backtracking (Day 37–40)

Goal:

Learn backtracking templates and pruning techniques.

Daily plan:

Day 37: N-Queens theory and constraints. Day 38: Rat in a Maze / path finding. Day 39: Sudoku solver. Day 40: combine with recursion practice.

Core concepts:

Choice tree, pruning, constraints propagation, using visited/state markers.

Practice targets:

Implement N-Queens solver and Sudoku backtracking.

Sample problems:

N-Queens, Word Search, Sudoku Solver.

Pitfalls:

Inefficient state copying; failing to backtrack state cleanly.

Phase 8: Hashing & HashMaps (Day 41–46)

Goal:

Use hash-based structures for frequency counting and $O(1)$ lookups.

Daily plan:

Days 41–42: HashMap/HashSet basics and collision concepts. Days 43–44: frequency maps & two-sum patterns. Days 45–46: sliding window + hashmap combined problems.

Core concepts:

Hash collisions, load factor, unordered_map vs map (ordered) differences, amortized $O(1)$.

Practice targets:

Solve 30 problems focusing on hash patterns and optimizations.

Sample problems:

Two Sum, Longest Consecutive Sequence, Subarray Sum Equals K.

Pitfalls:

Using hashed keys that are mutable/unhashable, ignoring worst-case collisions.

Phase 9: Stacks & Queues (Day 47–52)

Goal:

Learn LIFO/FIFO structures and monotonic stack/queue patterns.

Daily plan:

Day 47: stack basics and balanced parentheses. Day 48: next greater element monotonic stack. Day 49: queue & deque basics. Day 50–52: sliding window maximum using deque and practice problems.

Core concepts:

Monotonic stacks, using deque for windowed problems, amortized operations.

Practice targets:

Implement stack-based solutions and deque window maxima.

Sample problems:

Valid Parentheses, Next Greater Element, Sliding Window Maximum.

Pitfalls:

Not handling empty-stack conditions; incorrect window boundaries.

Phase 10: Linked List (Day 53–58)

Goal:

Master singly & doubly linked lists and pointer techniques.

Daily plan:

Day 53: node structure and basics. Day 54: reverse linked list (iterative/recursive). Day 55: detect cycle (Floyd). Day 56: merge two lists. Day 57–58: complex pointer problems (remove nth from end).

Core concepts:

Pointer manipulation, in-place list changes, sentinel/dummy nodes, fast & slow pointers.

Practice targets:

Implement core list APIs and solve medium problems using pointer tricks.

Sample problems:

Reverse Linked List, Merge Two Sorted Lists, Linked List Cycle, Remove Nth Node.

Pitfalls:

Losing node references during rewiring; off-by-one errors with indexing from tail.

Phase 11: Advanced Sorting & Searching (Day 59–62)

Goal:

Understand $O(n \log n)$ sorts and binary search applied to problem answers.

Daily plan:

Day 59: merge sort implementation & proof. Day 60: quick sort partitioning and pitfalls. Day 61: heap sort overview. Day 62: binary search on answer practice problems.

Core concepts:

Divide and conquer sorting, stability, pivot selection, median-of-three heuristic, external sort awareness.

Practice targets:

Implement merge & quick sort; solve median/allocate-books problems.

Sample problems:

Median of Two Sorted Arrays, Allocate Minimum Number of Pages (Book Allocation).

Pitfalls:

Bad pivot causing $O(n^2)$ quicksort; ignoring in-place merge complexities.

Phase 12: Trees & BST (Day 63–68)

Goal:

Master tree traversals, BST operations, and tree-based problem patterns.

Daily plan:

Day 63: tree basics & traversals (inorder, preorder, postorder). Day 64: level-order (BFS). Day 65: BST insert/delete/LCA. Day 66–68: problems on diameter, balanced tree check.

Core concepts:

Recursion on trees, subtree properties, height vs depth, lowest common ancestor methods.

Practice targets:

Solve 25 tree problems across traversals and structural queries.

Sample problems:

Binary Tree Inorder Traversal, Lowest Common Ancestor, Diameter of Binary Tree.

Pitfalls:

Confusing depth/height; not returning correct aggregated values from recursion.

Phase 13: Heaps & Priority Queues (Day 69–72)

Goal:

Use heaps for top-k queries and stream medians.

Daily plan:

Day 69: heap basics (min-heap vs max-heap). Day 70: top-K problems. Day 71: median of stream. Day 72: heap sort & applications.

Core concepts:

Heapify, push/pop complexities, customizing comparator, merging k-sorted lists.

Practice targets:

Implement top-k with heaps and maintain medians over sliding data.

Sample problems:

Kth Largest Element, Merge K Sorted Lists, Median from Data Stream.

Pitfalls:

Excessive memory use by copying arrays into heap; ignoring lazy deletion needs.

Phase 14: Graphs (Day 73–78)

Goal:

Understand graph representations and core algorithms (BFS, DFS, shortest paths, MST).

Daily plan:

Day 73: graph models and adjacency list vs matrix. Day 74: BFS/DFS basics and applications. Day 75–76: shortest path algorithms (Dijkstra, Bellman-Ford). Day 77: MST (Kruskal, Prim). Day 78: practice mixed graph problems.

Core concepts:

Directed vs undirected graphs, weighted edges, negative edges handling, cycle detection, topological sort.

Practice targets:

Solve 30 graph problems including connectivity, shortest paths, and MSTs.

Sample problems:

Course Schedule (topological sort), Number of Islands (BFS/DFS), Dijkstra shortest path.

Pitfalls:

Misusing adjacency matrix for sparse graphs (memory/time overhead); not resetting visited arrays.

Phase 15: Dynamic Programming (Day 79–85)

Goal:

Build DP thinking: identify subproblems, define state, transition, base cases, and optimize.

Daily plan:

Day 79: DP fundamentals, memoization vs tabulation. Day 80–81: classic 1D DP (Fibonacci, Climbing Stairs, House Robber). Day 82–83: 2D DP (LCS, unique paths, min path sum). Day 84–85: knapsack variants & partition DP.

Core concepts:

State definition, overlapping subproblems, optimal substructure, reconstructing solution from DP table.

Practice targets:

Solve 40 DP problems starting easy → medium → hard; focus on state design and transitions.

Sample problems:

Knapsack 0/1, Longest Increasing Subsequence (patience sorting intro), Unique Paths, Partition Equal Subset Sum.

Pitfalls:

Failing to reduce state size; missing memoization leading to TLE.

Phase 16: Greedy Algorithms (Day 86–88)

Goal:

Identify when greedy works and common greedy strategies.

Daily plan:

Day 86: activity selection & interval scheduling. Day 87: Huffman coding & greedy proof patterns. Day 88: MST revisit with greedy strategies.

Core concepts:

Greedy choice property, exchange argument for correctness, interval scheduling, earliest finish time heuristic.

Practice targets:

Implement 10 greedy problems and prove correctness informally.

Sample problems:

Activity Selection, Minimum Platforms, Huffman Encoding.

Pitfalls:

Applying greedy where DP is required; failing to justify greedy choice.

Phase 17: Tries & Advanced Topics (Day 89–90)

Goal:

Gain exposure to tries and optional advanced topics to round up your skillset.

Daily plan:

Day 89: tries implementation and word-search problems. Day 90: wrap-up optional topics: bit manipulation, union-find, segment trees, KMP/Z algorithm.

Core concepts:

Prefix trees, disjoint set union operations, range queries via segment trees, pattern matching automata.

Practice targets:

Implement a trie and use it to solve word-search style problems; attempt one union-find and one segment tree challenge.

Sample problems:

Implement Trie (insert/search), Word Search, Longest Common Prefix, KMP pattern matching.

Pitfalls:

Spending excessive time on niche topics without mastering core areas first.

Practice Strategy, Mock Interviews & Tracking

- Weekly routine: 5 practice days + 1 review day + 1 mock interview day.
- Mock interviews: start at day 45 with 30–45 minute simulations; increase to full-length mocks by day 75.
- Performance metrics: solve rate, success rate on medium/hard, time per problem, correctness on first attempt.
- Tracking template: date, problem name, platform, difficulty, time taken, mistakes, notes, resolved (Y/N).

Recommended Resources

- Practice platforms: LeetCode, Codeforces, InterviewBit, AtCoder.
- Reading & references: GeeksforGeeks, CP-Algorithms, CLRS (for algorithms theory).
- Visualizer tools: LeetCode Explore, Algorithm visualizations (online).
- Community: join a WhatsApp/Discord study group; discuss solutions and post weekly doubts.

Daily Checklist & Productivity Tips

- Warm-up: 10–15 minutes of easy problems to get into flow.
- Use a focused timer (Pomodoro: 25/5 or 50/10).
- After solving: write a 2–3 line summary of the pattern used.
- Once per week: revisit mistakes log and re-solve those problems.

Call to Action & How to Access Full Materials

- Join the WhatsApp channel for grouped study, full PDF collections, and weekly curated problem sets.
- Follow on Instagram/handle: @shreyaslarer for quick tips and carousel summaries.
- Suggested reading pack available: curated list of 100 essential problems mapped to this roadmap (ask for access in channel).

Appendix: Sample 14-day Intensive Plan (DP + Graphs focus)

This appendix provides a day-by-day micro-plan for two challenging topics: Dynamic Programming and Graphs. Day 1: DP fundamentals, memoization examples (Fibonacci), 2 practice problems. Day 2: 1D DP (Climbing Stairs, House Robber) and exercises. Day 3: LIS introduction with $O(n^2)$ DP and patience sorting overview. Day 4: LCS and string DP basics. Day 5: Grid DP (Unique Paths, Min Path Sum). Day 6: Knapsack 0/1 problem theory and practice. Day 7: Partition DP & Palindrome Partitioning intro. Day 8: Review week: re-solve all solved DP problems from memory. Day 9: Graph basics refresh and adjacency representations. Day 10: BFS/DFS problems and practice (island counting, bipartite check). Day 11: Dijkstra and priority-queue based shortest paths exercises. Day 12: Bellman-Ford and negative-weight handling problems. Day 13: MST algorithms (Kruskal, Prim) and union-find practice. Day 14: Mixed problem set + full mock interview focusing on DP/Graphs.

Prepared for students following the 90-day DSA roadmap. For customizations, curated problem lists, or slide/poster versions, request via the WhatsApp channel.